

Research For Taxi Tap
Taxi Tap by Git It Done



Contents

1	Introduction	3
2	Payment System	3
3	South African Taxi Industry Context	3
3.1	Route Registration and Ownership	3
3.2	Operational Constraints	3
4	Taxi Route Mapping Research	4
4.1	Data Acquisition	4
4.2	Data Cleaning and Transformation Process	4
4.2.1	PDF Parsing and Text Extraction	4
4.2.2	Address Normalization	4
4.3	Reverse Geocoding Implementation	5
4.4	Data Storage and Structure	5
4.5	Validation and Quality Assurance	6
5	Multi-Leg Journey Planning Research	6
5.1	Problem Definition	6
5.2	Intersection Detection Algorithm	6
5.3	Transfer Point Eligibility	7
5.4	Journey Construction	7
5.5	Implementation Evolution	7
5.6	Visualization and Verification	8
6	Technical Challenges and Solutions	9
6.1	PDF Parsing Complexity	9
6.2	Geocoding Accuracy	10
6.3	Performance Optimization	10
7	Research Impact on Application Design	10
7.1	Route-Centric User Interface	10
7.2	Transfer Guidance	10
7.3	Cash Payment Integration	10
7.4	Scalability Considerations	11
8	Future Research Directions	11
9	Conclusion	11

1 Introduction

During the course of this project, Taxi Tap, we had to do quite a bit of research to ensure that we have an application that is well thought out and takes both passengers and drivers into consideration. In the following, we discuss the sections of research we had to do to enhance our system and ensure that users are satisfied.

2 Payment System

As you will see when using the application, we do not have a digital payment system integrated into our mobile application. This is because after doing user research we came to the conclusion that a digital payment system is not ideal for the taxi system we have today.

After speaking to multiple individuals who are taxi passengers, we understood that the passengers, as well as the drivers, are comfortable with the cash payment system. Thus for our system we decided to use this information to our advantage.

Since there is already an established trust between passengers and drivers, we decided to enhance our application by integrating some sort of cash payment system. This payment system focuses on helping the passenger (the one who sits at the front), to easily make sure everyone gets enough change and that all passengers have paid the correct amount. The driver can also see payments that are in the waiting state and passengers that have not paid to ensure that drivers get the most out of our application.

3 South African Taxi Industry Context

The South African minibus taxi industry operates under a unique system that greatly influenced how we designed Taxi Tap. Understanding how this industry works was crucial for building a system that works with the existing structure, not against it.

3.1 Route Registration and Ownership

In South Africa, taxi routes are registered with taxi associations that control specific areas and routes. Multiple associations can run similar routes, creating a complex network. Drivers pay fees to join associations, which then assign them to specific routes. This assignment is fixed—drivers cannot change their routes to pick up passengers who need to go elsewhere.

3.2 Operational Constraints

Unlike other ride-hailing applications where drivers can go anywhere, South African taxi drivers must stick to their assigned routes. Passengers cannot ask for door-to-door service to any location. Instead, passengers must understand the route network and plan trips that match available routes. This created our main problem: how can passengers easily plan trips across multiple fixed routes?

This research led to a key feature: smart multi-leg trip planning for destinations that need more than one taxi.

4 Taxi Route Mapping Research

Accurate route information is the foundation of Taxi Tap’s trip planning. Without knowing where taxis go and where they stop, passengers cannot plan their trips. This section explains how we got, processed, and organized taxi route data.

4.1 Data Acquisition

We got route data from official government documents containing about 100 route registrations submitted by taxi associations. These PDF documents showed the legal routes that drivers can operate.

These documents had legitimate route information but were challenging to work with. Directions were written in everyday language, mixing official addresses with local place names that aren’t in standard map databases. The format also varied a lot between different associations, with no standard template.

4.2 Data Cleaning and Transformation Process

Turning raw PDF documents into structured, geocoded route data needed a complete data science pipeline built in Python. This process followed the full data analysis cycle: getting data, cleaning it, transforming it, enriching it, and validating it.

4.2.1 PDF Parsing and Text Extraction

The first challenge was getting usable text from PDF documents with different formats. Using Python’s text extraction libraries, we built a parser that could handle the inconsistent structures. The main libraries we used were:

- `re` for pattern matching to find route structures
- `requests` for API calls to geocoding services
- `json` for data serialization
- Standard Python logging to monitor the process

The parser found route names, stop sequences, and directions from the unstructured text. This needed pattern recognition rules that could adapt to format changes while staying accurate.

4.2.2 Address Normalization

After extracting text, the next challenge was normalizing the different address formats. Directions included:

- Official street addresses
- Local place names

- Landmark references
- Informal location descriptions

We added logic to handle common variations, like different spellings or multiple names for the same location. Luckily, many informal locations had been added to map databases by South African contributors, though as unofficial listings. This community data was very helpful for figuring out unclear location references.

4.3 Reverse Geocoding Implementation

Converting text descriptions to exact latitude and longitude coordinates needed good geocoding. We used a two-part approach:

1. **Primary Service:** OpenStreetMap’s Nominatim API was our main geocoding service because it has good coverage of South African locations and generous usage limits.
2. **Fallback Service:** For locations that failed with OpenStreetMap, we tried Google Maps Geocoding API, using the limited free tier for difficult cases.

This approach achieved over 90% success. For the remaining failures, we used the stop coordinates as a reference point and kept the original stop name for passenger recognition.

The geocoding script included rate limiting and error handling to manage API quotas:

```
import time
import requests
from typing import Optional, Tuple

def geocode_location(location: str) -> Optional[Tuple[float, float]]:
    # Primary: OpenStreetMap
    # Fallback: Google Maps
    # Rate limiting and retry logic
```

4.4 Data Storage and Structure

Successfully geocoded routes were first stored as JSON files, which made debugging and validation easier. Each route object contained:

- Route identifier and name
- Associated taxi association
- Ordered list of stops with coordinates
- Metadata like estimated duration and base fare

This structured data was then moved to our backend database, where it powers real-time route queries and trip planning in the live application.

4.5 Validation and Quality Assurance

One limitation of our approach was not having systematic validation. We assumed that government-submitted route registrations were accurate, as these represent the legal routes associations can operate. While this allowed us to develop quickly, field validation or crowdsourced verification would be good future improvements.

5 Multi-Leg Journey Planning Research

Many destinations cannot be reached by a single route. Passengers must transfer between routes run by different associations, often walking between stops. This section describes how we built trip planning across multiple routes.

5.1 Problem Definition

The multi-leg trip problem is: Given a starting point O and destination D, find a sequence of taxi rides and walking that connects O to D, where each taxi ride must follow a registered route and walking distances must be reasonable.

This is different from normal route planning because:

- Routes are fixed and cannot change
- Transfers must happen at or near route stops
- Multiple associations work independently
- Walking between stops must be reasonable for passengers

5.2 Intersection Detection Algorithm

The basis of multi-leg planning is finding where different routes cross or come close enough to allow transfers. We built a Python algorithm using geospatial analysis libraries:

```
import pandas as pd
import networkx as nx
from typing import List, Dict, Tuple, Set
import math
```

The algorithm calculates distances between all stops across different routes using the Haversine formula, which finds distances between coordinate pairs on a sphere. For each pair of stops from different routes, if the distance is below 3 kilometers, the stops are marked as possible transfer points.

We chose this threshold to represent a reasonable walking distance for passengers: about 30 to 45 minutes of walking at average pace. While longer than ideal, it reflects the reality of sparse route coverage in some areas and the need for transfers to reach certain destinations.

5.3 Transfer Point Eligibility

Not all crossings make good transfer points. Our algorithm checks several things:

- **Distance:** Stops must be within 3 kilometers
- **Route Direction:** The sequence must make sense (origin route goes toward the transfer point, destination route goes from transfer point toward destination)
- **Association Diversity:** Transfers should connect routes from different associations, as same-association routes usually have direct connections

The algorithm builds a transfer graph where nodes are route stops and edges are valid transfers, weighted by walking distance and estimated time.

5.4 Journey Construction

Given origin and destination coordinates, the multi-leg planner:

1. Finds routes with stops near the origin (within 1 kilometer walking distance)
2. Finds routes with stops near the destination (within 1 kilometer)
3. Dynamically finds intersecting points between combinations of origin and destination routes
4. Checks that stop sequences respect route order (no backtracking)
5. Calculates total cost and estimated time for each possible trip
6. Returns the top options ranked by total cost

Currently, the system supports trips with up to two legs (one transfer). Future improvements could extend this to three-leg trips for better coverage.

5.5 Implementation Evolution

The multi-leg planning system was first built in Python using libraries like `pandas` for data work, `networkx` for graph algorithms, and `folium` for visualization. This prototype validated the approach and let us experiment with different distance thresholds and ranking criteria.

The production version was then built in TypeScript for integration with our backend. The TypeScript version uses the same algorithm while connecting with our real-time database and giving fast responses to mobile clients. Key parts include:

- Distance calculation using the Haversine formula
- Dynamic fare calculation based on distance traveled
- Route filtering by driver availability
- Transfer point detection with adjustable distance thresholds
- Trip validation ensuring proper stop order

5.6 Visualization and Verification

To verify our intersection detection and trip planning worked correctly, we created visualizations using Python’s `folium` library, which makes interactive maps. These visualizations helped us:

- Verify that detected intersections were geometrically correct
- Find potential problems with route data quality
- Show multi-leg trip options to stakeholders
- Provide a reference for future improvements

Figure 1 shows the mapped route network with all stops visualized, showing the coverage we achieved through our data processing. Figure 2 highlights detected transfer points between routes, showing where passengers can switch between different taxi services. Figure 3 shows the complete network with both stops and transfer points, showing the full complexity of the multi-leg trip planning problem.

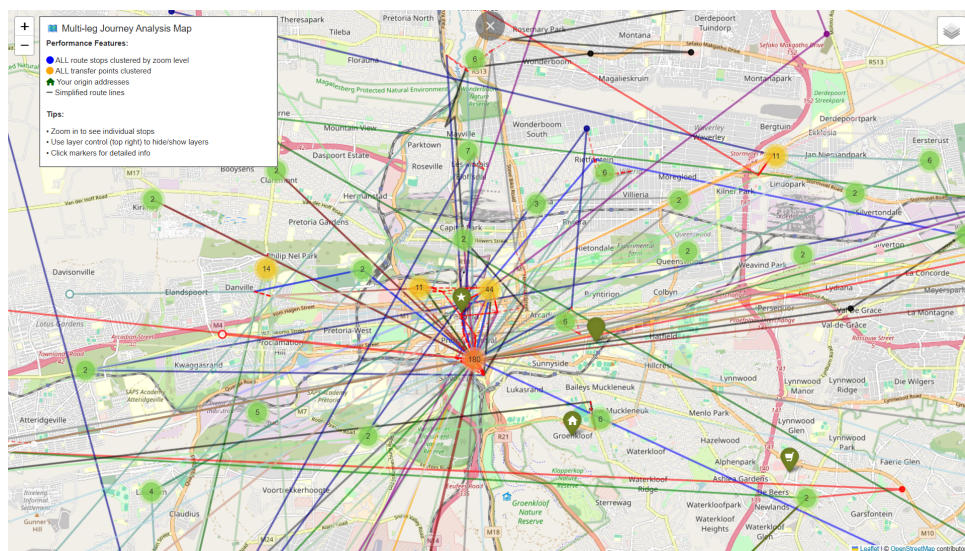


Figure 1: Mapped taxi routes showing all geocoded stops across the service area

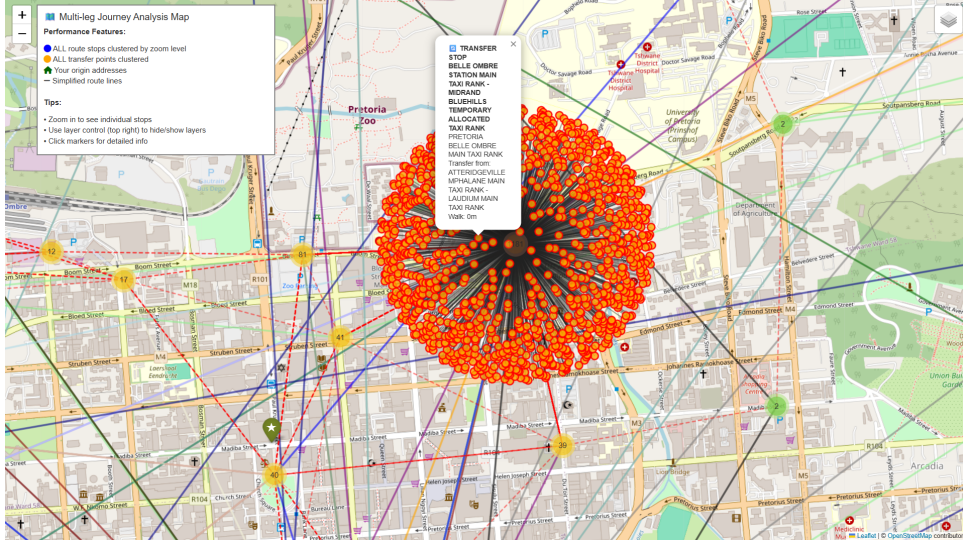


Figure 2: Detected transfer points where passengers can switch between routes

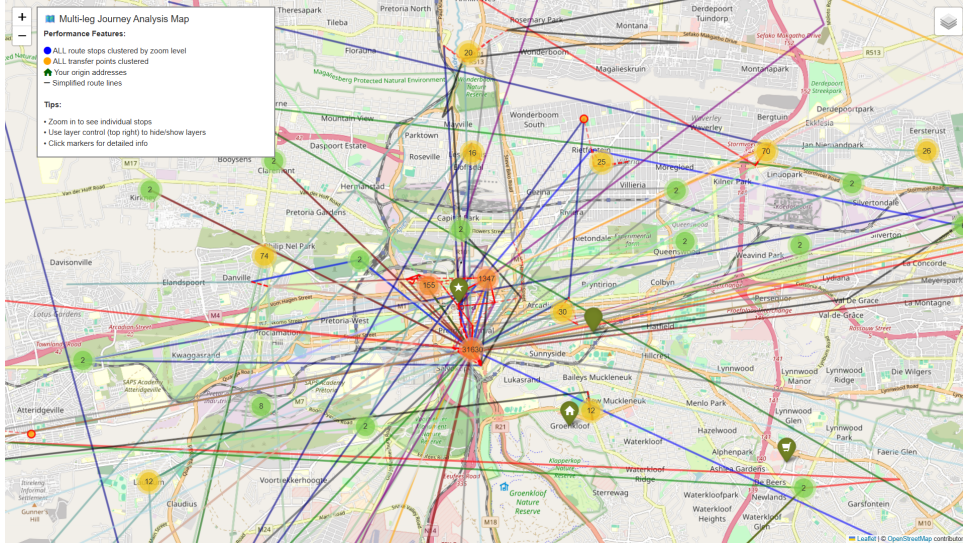


Figure 3: Complete network view showing routes, stops, and transfer points

6 Technical Challenges and Solutions

During research and development, several big challenges came up that needed solutions.

6.1 PDF Parsing Complexity

The main technical challenge was getting structured information from inconsistently formatted PDF documents. Different taxi associations submitted route information using different templates, naming styles, and structures. Our solution was building flexible parsing rules that could find key elements (route names, stop sequences, directional indicators) regardless of document structure, while logging unclear cases for manual review.

6.2 Geocoding Accuracy

Getting high geocoding accuracy for local and informal location names was difficult. Many stops are known locally by names that don't appear in official map databases. Our approach using both OpenStreetMap and Google Maps as fallback greatly improved success rates. Also, using community-contributed geographic data in Google Maps was very helpful for South African-specific location references.

The remaining geocoding failures (less than 10% of stops) were handled by using stop coordinates as position anchors while keeping human-readable names for passenger recognition.

6.3 Performance Optimization

Computing all possible multi-leg trips across 100 routes with thousands of stops created performance issues. The transfer points are calculated in real-time in the application code using the Haversine distance formula between stops from different routes. To optimize this, routes are first filtered by driver availability in the trip area before computing transfer points, which greatly reduces the number of calculations needed. The graph visualizations created during development were mainly used for testing and verification rather than as a pre-computed lookup structure.

7 Research Impact on Application Design

The research in this document directly shaped Taxi Tap's main features and user experience.

7.1 Route-Centric User Interface

Understanding that routes are fixed and drivers cannot change them informed our decision to show route information clearly in the user interface. Rather than hiding route complexity, we show it, telling passengers which routes serve their trip and where they need to board and get off.

7.2 Transfer Guidance

The multi-leg trip planner translates complex network analysis into simple, clear guidance for passengers. Users get step-by-step instructions: which route to take first, where to transfer, how far to walk, which route to take next, and the total estimated cost. This moves the burden of network planning from the passenger to the application.

7.3 Cash Payment Integration

As discussed in Section 2, our research into payment preferences showed strong preference for cash transactions. This finding, combined with our route and trip planning research, led to a complete design that respects existing industry practices while adding modern conveniences.

7.4 Scalability Considerations

The data processing pipeline is designed for growth. As more routes are registered or existing routes change, the Python scripts can be run again to update the route database and transfer point graph. This ensures Taxi Tap can grow with the evolving taxi industry without needing major changes.

8 Future Research Directions

While our current research provides a solid foundation for Taxi Tap, several areas need future work:

- **Ground Truth Validation:** Systematic validation of route data through field studies or driver surveys
- **Real-Time Route Updates:** Integration with taxi association systems to get automatic route updates
- **Three-Leg Journeys:** Extending multi-leg planning to support more transfers for better coverage
- **Dynamic Transfer Points:** Adding real-time traffic and safety data to recommend best transfer locations
- **Crowdsourced Corrections:** Letting drivers and passengers suggest route corrections or additional stops
- **Predictive Analytics:** Using historical trip data to estimate wait times and best departure times

9 Conclusion

The research done for Taxi Tap shows the value of understanding the domain before building technical solutions. By thoroughly investigating the South African taxi industry's unique operational model, we built a system that works within existing constraints rather than fighting against them. The data science pipeline for route mapping and the algorithm for multi-leg trip planning are significant technical contributions that make taxi travel more accessible and understandable for passengers while respecting the established industry structure.